CS598APE Due: **November 9, 2026**

Mini-Paper 3: Abelian Sandpile Model

Name: Don't forget to add your name *Collaborators: List all sources used*

The final mini-paper this semester is to analyze and improve the end to end performance of a 2D finite Abelian Sandpile. The scope of this homework is designed to be smaller than the other 2 homeworks, in order to let you focus more on your projects.

For this project, you will work **individually**, and the grading will be based on your report (section 4.1) and your artifact evaluation (section 4.2). You will also peer review the reports and artifacts of others.

You are welcome to use any source of inspiration or tool to create your optimizations, subject to the course policy on the website. These may include new code/changes to code and the use of different tools to generate or change code. You are welcome to also try more advanced techniques that do not fall into this category (e.g. specific hardware). For precise details, check section 4.2.

1 Accessing Course Infrastructure

You are welcome to use any hardware of your choosing to test and perform optimizations, assuming that the optimizations you write can be replicated by others. This does not mean that others need to replicate your exact performance results (e.g. exactly 12% boost by doing X), but that the optimization you write generally results in an improvement and does not rely on non-standard hardware features. If you want to use your own machine for setup and optimization, you are responsible for ensuring the requisite requirements are available.

We have given students access to UIUC teaching VMs. You may use them for completing the homeworks and project.

2 Abelian Sandpile Model

An abelian sandpile of size $N \times N$ is a grid containing the number 0,1,2 and 3 (representing the number of grains of sand). Adding two sandpiles works by first adding element by element, and then *toppling* any element that goes over 3. A toppling removes four grains from that cell and sends one to each of its four direct neighbors. Grains that fall off the grid disappear.

The order in which you topple doesn't matter, since the end result always remains the same (hence the term **abelian**). We write $\mathrm{stab}(S)$ for the stable sandpile obtained by toppling until every cell has at most three grains, and define sandpile addition by $A \oplus B = \mathrm{stab}(A + B)$, where $+$ is elementwise addition. For example, add the following two 3×3 sandpiles and stabilize:

$$\begin{pmatrix} 3 & 3 & 3 \\ 3 & 3 & 3 \\ 3 & 3 & 3 \end{pmatrix} + \begin{pmatrix} 1 & 2 & 1 \\ 2 & 1 & 2 \\ 1 & 2 & 1 \end{pmatrix} = \begin{pmatrix} 4 & 5 & 4 \\ 5 & 4 & 5 \\ 4 & 5 & 4 \end{pmatrix} \xrightarrow{\text{stabilize}} \begin{pmatrix} 2 & 1 & 2 \\ 1 & 0 & 1 \\ 2 & 1 & 2 \end{pmatrix}.$$

The arrow summarizes a sequence of topplings.

We are interested in the *recurrent* sandpiles: the stable configurations that can be obtained by adding grains to the all-3s grid and stabilizing. These form a group G under $\oplus$.

Any group G (it can be numbers, vectors, or in this case the recurrent sandpiles) has a special element called the identity element. The identity element e_N has the following property – for any element $x \in G$

$$x \oplus e_N = x$$

In the case of the integer set $\mathbb{Z}$, we have $\oplus = +$ (just addition), and the identity element is just the number **0**.

Similarly, *recurrent sandpiles* form a group, which has a unique identity. For the 3×3 case, this identity sandpile is just

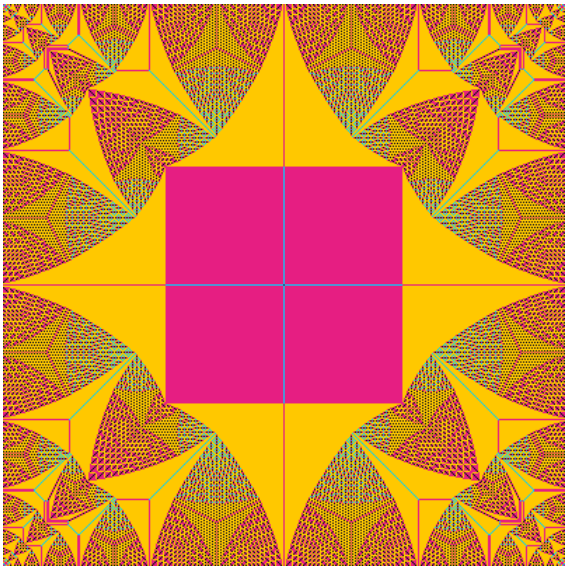
$$e_3 = \begin{pmatrix} 2 & 1 & 2 \\ 1 & 0 & 1 \\ 2 & 1 & 2 \end{pmatrix}.$$

For example, adding e_3 to the all-2s sandpile leaves it unchanged after stabilization:

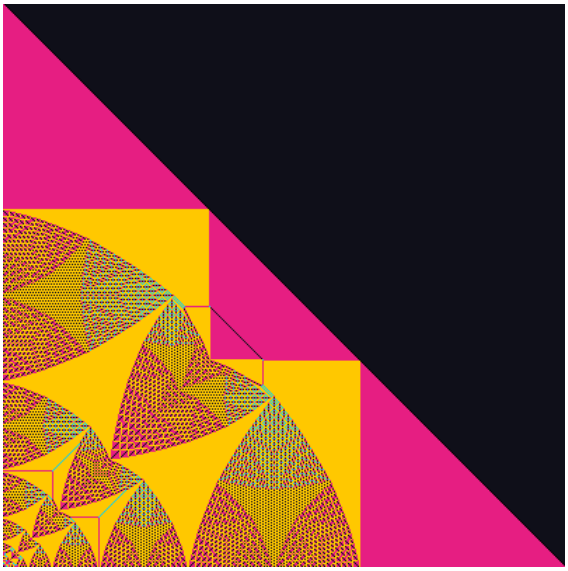
$$\begin{pmatrix} 2 & 2 & 2 \\ 2 & 2 & 2 \\ 2 & 2 & 2 \end{pmatrix} + e_3 = \begin{pmatrix} 4 & 3 & 4 \\ 3 & 2 & 3 \\ 4 & 3 & 4 \end{pmatrix} \longrightarrow \begin{pmatrix} 0 & 5 & 0 \\ 5 & 2 & 5 \\ 0 & 5 & 0 \end{pmatrix}$$
$$\longrightarrow \begin{pmatrix} 2 & 1 & 2 \\ 1 & 6 & 1 \\ 2 & 1 & 2 \end{pmatrix} \longrightarrow \begin{pmatrix} 2 & 2 & 2 \\ 2 & 2 & 2 \\ 2 & 2 & 2 \end{pmatrix}.$$

The arrows topple the four corners, the four edge centers, and finally the center.

Identity sandpiles show some interesting symmetric structure when you scale N upto very high values. For example, this is the identity sandpile for a 401×401 grid of values. Black denotes 0 grains, Blue denotes 1 grain, Pink denotes 2 grains and Gold denotes 3 grains.



We can also change the shape of the underlying grid to be something other than a square. We still see the appearance of structure (e.g a triangular grid).



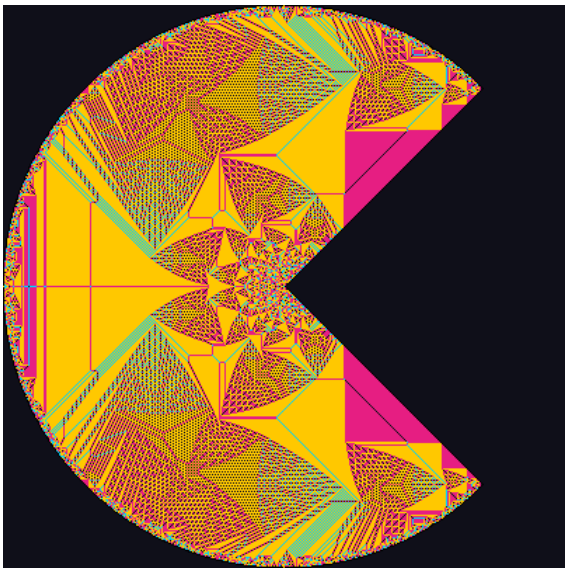
2.1 Where Is the Code?

The application code for this project is available on GitHub. Fork this code, but please keep your code and optimizations in private (and not public) repositories.

The README.md contains more information about the layout of the symbolic regression code and how to execute it.

2.2 What is your Task?

For this homework, you will be expected to find the identity sandpile for a 1001×1001 Pacman-esq grid. It looks like this



3 How to Tackle this Homework

1. Create a private fork of the homework repository on GitHub. Instructions on how to do that are in this GitHub gist.
2. Look at the code, see where you can make some obvious improvements. Profile, identify bottlenecks, and optimize them. Keep doing this again and again. Don't prematurely optimize.
3. Use `perf` to identify performance hotspots in your code. One super easy way to get started is to use `perf` to produce a flamegraph, and see the busiest functions in your code.
4. Use debuggers like `gdb` or `rr` to assist you with debugging. Ensure that your programs are safe from memory leaks using tools like `valgrind`.
5. Implement one optimization at a time. This helps in reasoning about your optimizations, and reproducing the effect of a specific optimization if needed.
6. Document your approach in the report. Make sure your code is reproducible (the whole point of `Docker` is to avoid the *oh but it runs on my computer...*)
7. Review & run the code your peers write!

4 What to Hand In

4.1 Report

Your report should be about (and no more than) 3 pages long(excluding references). Use the ACM Sigplan Template (follow the formatting in `sample-sigplan.tex`). It should include:

1. A **brief description** of your optimizations. This can include optimizations which helped performance, optimizations which did not help performance, or potential optimizations you did not implement and why (for example as profiling indicated it not to be helpful). Describe what kind of optimization you tried (e.g. algorithmic, data-structure, parallelism, caching, work-reduction, etc).
2. **Specify the assumptions** which an optimization relies on, and the scope in which your optimization applies. For example, one could hardcode the application to directly output the desired image (perhaps conditional on the exact input specification) – but this would only provide a speedup for that scene at that resolution.
3. **Include graphs** to indicate performance improvements/slowdowns (if any). Remember, the goal here is to not just get blind speedups, but also attempt to investigate why we are getting those speedups (or perhaps why we are not).
4. A brief **introspection** section for your optimizations: Performance engineering involves coming up with ideas that can improve performance, and profiling/deciding which ones will be impactful in practice. Sometimes you can do something but speedup up a function $100\times$ that represents $< 1\%$ of runtime won't change much. Similarly, sometimes you need to do prerequisite optimizations before the fruit of a bigger optimization can apply (e.g. rewrite a datastructure to be a linear scan is required before caching can be improved).

4.2 Artifact

You will be expected to submit an artifact, which will be independently evaluated by random team. To ensure the ease of reproducibility, the artifact should be a well documented git repo which should have at-least the following things in it's README.md

1. **All the tools needed to construct a binary from your source code**. We recommend modifying `docker/Dockerfile` and specifying the packages needed to install/build your tools.
2. **Instructions on how to build and run** your code. This includes how to run and benchmark your baseline code, and verify that your optimizations hold water. Remember, part of your grade depends on whether your peers are able to replicate the results stated in your paper.
3. **List your optimizations, and what commands, flags, or earlier git commits** are needed to evaluate their effectiveness.

Custom Hardware You are welcome to also try more advanced optimizations that require specific hardware to run – for example GPU tensor cores, RTX, DLSS, Intel AVX-512 or Arm NEON intrinsics(for the M1 Mac folks...).

However, **in this case you must submit two artifacts**: one artifact which can be run by anyone with a standard Unix machine (like the provided course VMs or Docker files), and a second artifact detailing optimizations based on non-replicable setups. We recommend using separate git branches or commit hashes to easily distinguish both artifacts.

5 How to Submit?

You will be using HotCRP to submit and peer-review your reports. We will open up the submission link as soon as the deadline nears. As for the code, it is sufficient to provide a link to the github repo you will be using for writing the code.